

Machine Learning

From Zero to Research Level

A complete, step-by-step self-study roadmap
for **classical** ML — before Deep Learning & Neural Networks

What this document is. A 52-week plan (about 10 hours per week) that takes you from programming and mathematics fundamentals, through every core classical ML algorithm, into statistical learning theory and probabilistic ML, and finishes with real research skills: reproducing papers, running rigorous experiments, and writing a paper in $\text{\LaTeX}$.

What comes next. Once you pass the exit criteria in the final section, you are ready for a separate Deep Learning & Neural Networks roadmap.

Contents

1	The Big Picture	3
2	How to Study (Read This First)	4
3	Phase 0 — Setup and Programming Foundations	5
4	Phase 1 — Mathematical Foundations	7
5	Phase 2 — Core Machine Learning	10
6	Phase 3 — Advanced Classical ML and Theory	14
7	Phase 4 — Research Skills	17
8	Projects Across the Roadmap	21
9	Master Resource List	22
10	Ready for Deep Learning and Neural Networks?	25
A	Master Week-by-Week Schedule	27
B	Suggested Repository Structure	29
C	Weekly Log Template	29
D	Cold-Email Template for Research Groups	29
E	From-Scratch Implementation Checklist	30

1 The Big Picture

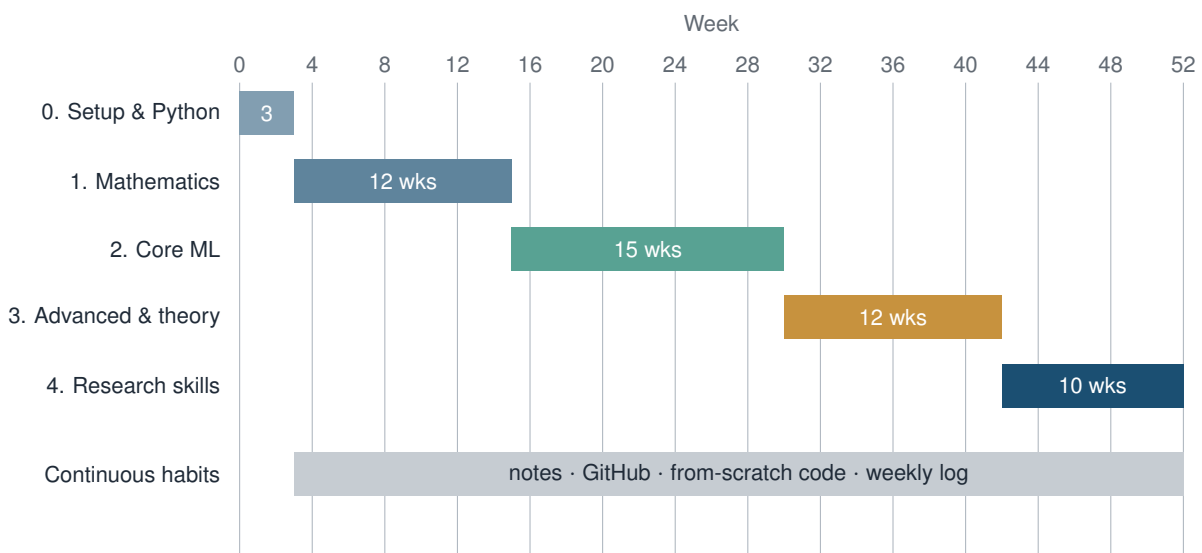
1.1 What “research level” actually means

Research level in ML does not mean “knows many libraries”. It means you can do five things independently:

1. **Understand** an algorithm well enough to derive it on paper and implement it from scratch.
2. **Read** a current paper, identify its claim, its assumptions, and where it could be wrong.
3. **Reproduce** published results and explain any discrepancy you find.
4. **Design** experiments that actually answer a question (baselines, ablations, statistics).
5. **Communicate** findings clearly in a paper-style $\text{\LaTeX}$ report.

Every phase below trains one or more of these abilities, and every phase ends with exit criteria that test them.

1.2 Timeline at a glance



Phase	Weeks	Outcome
0. Setup & Python	1–3	Working toolchain, Git, NumPy/pandas/Matplotlib fluency, first $\text{\LaTeX}$ note.
1. Mathematics	4–15	Linear algebra, calculus, probability, statistics, optimisation, information theory — at the level ML actually needs.
2. Core ML	16–30	Every standard supervised and unsupervised algorithm, derived and implemented from scratch; a full end-to-end project.
3. Advanced & theory	31–42	Learning theory, convex optimisation, Bayesian ML, graphical models, sparsity, bandits, uncertainty, causality primer.
4. Research skills	43–52	Paper reading, reproduction, rigorous experiments, a mini research project, a written paper draft, internship outreach.

1.3 Assumptions and how to adapt the plan

- **Time budget:** about 10 hours per week alongside your degree. During semester breaks, double the hours and finish a phase in about half the weeks.
- **Starting point:** each phase opens with a *skip test*. If you can pass it honestly, skip straight to the next phase.
- **If you fall behind:** cut breadth, never depth. Dropping an optional topic is fine; skipping the from-scratch implementations is not.
- **Exam weeks:** take a planned pause of up to two weeks, then resume where you stopped. Do not try to “catch up” by cramming two weeks into one.

2 How to Study (Read This First)

2.1 The learning loop

Use the same five-step loop for every topic in this document:

1. **Learn** — watch the lecture or read the chapter once, actively, with a pen.
2. **Derive** — close the material and re-derive the key result on paper.
3. **Implement** — code it from scratch in NumPy (no ML library) and test it against `scikit-learn`.
4. **Apply** — run it on at least one real dataset and look at the failures.
5. **Explain** — write a half-page L^AT_EX note in your own words as if teaching a classmate.

Common pitfall: passive learning

Watching 40 hours of video feels productive but produces little durable skill. A rule of thumb: for every hour you watch or read, spend at least one hour deriving, coding or solving problems.

2.2 Weekly rhythm (about 10.5 hours)

Session	Length	What to do
Mon, Wed, Fri	1.5 h each	Theory: lecture or book chapter, with handwritten notes and worked examples.
Tue, Thu	1.5 h each	Coding: implement the week’s algorithm; run experiments; debug against <code>scikit-learn</code> .
Saturday	2 h	Project block: the current deliverable, Kaggle-style practice, or paper reproduction.
Sunday	1 h	Review: re-do two problems you got wrong, update your error log, write the weekly log entry (template in Appendix C).

2.3 Your toolkit (set up in Week 1)

- ☐ **GitHub repository** named `ml-journey` with folders for notes, code, projects and papers (structure in Appendix B).
- ☐ **Python environment** managed with `conda` or `venv`, pinned in a `requirements.txt` or `environment.yml`.
- ☐ **Jupyter / VS Code** for exploration; plain `.py` modules for reusable from-scratch implementations.
- ☐ **Overleaf or a local L^AT_EX install** for all written notes, so L^AT_EX becomes second nature before you write a paper.
- ☐ **Reference manager** (Zotero) from the first paper you read.
- ☐ **Error log** (a simple markdown file): every mistake you make in a derivation or bug in your code, with the cause. Review it every Sunday.

2.4 Rules that keep the plan honest

1. **No black-box shortcuts first.** Implement from scratch before you use the library version.
2. **Test everything.** Compare your implementation to `scikit-learn` on the same data; they should agree to within numerical tolerance.
3. **Never tune on the test set.** Decide on a protocol (train / validation / test, or nested CV) before you look at results.
4. **Everything lives in Git.** Commit at least four times a week. A public repository is also your portfolio for internship applications.
5. **Exit criteria are strict.** If you cannot meet them, repeat the weakest topic for a week rather than moving on with gaps.

3 Phase 0 — Setup and Programming Foundations

Weeks 1–3

about 30 hours

Goal

Build the working environment and the minimum programming fluency that every later phase assumes: Linux shell, Git, Python, NumPy, pandas, Matplotlib, and basic L^AT_EX.

Skip test for Phase 0

Skip this phase if you can, without looking anything up: (a) create a virtual environment and install packages, (b) branch, commit, push and open a pull request on GitHub, (c) reshape and plot a pandas DataFrame, and (d) write a vectorised NumPy function with broadcasting (no Python loops over rows).

3.1 Week 1 — Shell, Git, environment

- ☐ Learn the essential shell commands: `cd`, `ls`, `cp`, `mv`, `rm`, `grep`, `cat`, pipes, redirects, and environment variables. A good free course is MIT's *The Missing Semester of Your CS Education*

(<https://missing.csail.mit.edu>).

- ☐ Install Python 3.11 or newer with conda or venv. Create one environment named ml. Install: numpy, pandas, matplotlib, seaborn, scipy, scikit-learn, jupyter.
- ☐ Learn Git: init, add, commit, status, log, diff, branch, merge, push, pull. Practise one merge conflict on purpose so it does not frighten you later.
- ☐ Create the GitHub repository ml-journey with the structure in Appendix B and a README.md describing your goal.
- ☐ Export your environment (pip freeze > requirements.txt or conda env export) and commit it.

Deliverable: repository online, environment reproducible from a fresh clone.

3.2 Week 2 — Python core and NumPy

- ☐ Python essentials: data types, list/dict/set comprehensions, functions with default arguments, classes and `__init__`, modules, exceptions, file I/O, pathlib.
- ☐ Write small functions with docstrings and simple tests (use assert first, then pytest).
- ☐ NumPy: array creation, shapes, dtypes, indexing and slicing, boolean masks, **broadcasting**, axis arguments, `np.linalg`, random number generation with a fixed seed (`np.random.default_rng(0)`).
- ☐ Rewrite every loop-based computation you wrote as a vectorised one. Time both versions with `timeit`.
- ☐ Do at least 30 problems from a public NumPy exercise set (for example the “100 NumPy exercises” collection).

Deliverable: a module `utils.py` with five tested helper functions (e.g. standardise columns, pairwise distances without loops, train/test split, one-hot encode, accuracy).

3.3 Week 3 — pandas, plotting, first L^AT_EX note

- ☐ pandas: `read_csv`, `loc/iloc`, `groupby`, `merge`, `pivot_table`, handling missing values, datetime columns, `apply` vs. vectorised operations.
- ☐ Matplotlib and seaborn: histograms, scatter plots, box plots, heatmaps, and pair plots. Label every axis; add titles and units.
- ☐ Do a complete exploratory data analysis (EDA) on one tabular dataset (UCI Machine Learning Repository, OpenML, or Kaggle). Ask: What are the distributions? Which values are missing, and why? Which features correlate? What looks like an outlier or error?
- ☐ L^AT_EX basics on Overleaf: sections, equations, figures, tables, `amsmath`, BibTeX. Write a one-page note summarising your EDA, with at least one figure and one equation.

Deliverable: EDA notebook and a one-page L^AT_EX PDF, both committed.

Exit criteria — do not move on until all are true

- ☐ You can set up a clean environment and project from scratch in under 15 minutes.
- ☐ You can load, clean, summarise and visualise an unfamiliar CSV file without a tutorial.

- ☐ You can explain broadcasting with an example and avoid explicit loops over rows.
- ☐ Your repository has at least 12 commits and a readable README.

4 Phase 1 — Mathematical Foundations

Weeks 4–15

about 120 hours

Goal

Learn exactly the mathematics that classical ML uses: linear algebra, multivariate calculus, probability, statistics, optimisation and information theory. The target is *fluency*: you should be able to derive results, not just recognise them.

Skip test for Phase 1

Skip a block only if you can, on paper: (LA) compute the SVD structure of a small matrix and explain what it means geometrically; (Calc) differentiate $f(\mathbf{w}) = \frac{1}{2} \|X\mathbf{w} - \mathbf{y}\|^2$ with respect to $\mathbf{w}$; (Prob) derive the posterior of a Beta–Bernoulli model; (Stat) write the MLE of a Gaussian’s mean and variance and show the variance estimator is biased.

Common pitfall: “math first, forever”

Do not postpone all ML until you finish “all the math”. This plan front-loads the core math for 12 weeks, then revisits each area inside Phases 2 and 3 where it is used. Your math will deepen as you apply it.

4.1 Weeks 4–5 — Linear algebra I: vectors, matrices, linear systems

Learn:

- Vectors, dot product, norms ($\ell_1, \ell_2, \ell_\infty$), angles, cosine similarity.
- Matrices as linear maps; matrix multiplication; transpose; inverse; determinant (intuition only).
- Linear systems, Gaussian elimination, LU factorisation, rank, column space, null space.
- Linear independence, span, basis, dimension.

Resources: Gilbert Strang, MIT 18.06 (lectures 1–10); 3Blue1Brown *Essence of Linear Algebra*; *Mathematics for Machine Learning* (Deisenroth, Faisal, Ong), Chapter 2.

Practice: 15–20 problems per week from Strang’s problem sets.

Deliverable: implement Gaussian elimination and back-substitution in NumPy; verify against `np.linalg.solve` on 50 random systems.

4.2 Weeks 6–7 — Linear algebra II: geometry, eigen-structure, SVD

Learn:

- Orthogonality, projections, orthonormal bases, Gram–Schmidt, QR factorisation.

- Least squares as projection: the normal equations $X^\top X \mathbf{w} = X^\top \mathbf{y}$.
- Eigenvalues and eigenvectors, diagonalisation, the spectral theorem for symmetric matrices.
- Positive (semi-)definite matrices and why they matter (covariance matrices, convexity, kernels).
- Singular value decomposition, low-rank approximation (Eckart–Young), pseudoinverse.

Resources: Strang 18.06 (lectures 11–30, selectively); MML Chapters 3–4; Trefethen & Bau, *Numerical Linear Algebra* (optional deep dive).

Derive by hand: (i) projection onto a subspace, (ii) normal equations from least squares, (iii) the SVD from the eigendecomposition of $A^\top A$.

Deliverable: implement power iteration for the top eigenvector, then PCA via SVD from scratch; reproduce `sklearn.decomposition.PCA` results on a toy dataset.

4.3 Weeks 8–9 — Multivariate calculus and matrix calculus

Learn:

- Derivatives, chain rule, partial derivatives, gradient, directional derivative, Jacobian, Hessian.
- Taylor expansion to first and second order; critical points; saddle points.
- Convex functions: definition, first- and second-order conditions.
- Matrix calculus identities, for example

$$\nabla_{\mathbf{x}}(\mathbf{a}^\top \mathbf{x}) = \mathbf{a}, \quad \nabla_{\mathbf{x}}(\mathbf{x}^\top A \mathbf{x}) = (A + A^\top) \mathbf{x}, \quad \nabla_{\mathbf{w}} \frac{1}{2} \|X \mathbf{w} - \mathbf{y}\|^2 = X^\top (X \mathbf{w} - \mathbf{y}).$$

- Gradient descent from first principles: step size, convergence on quadratics, effect of the condition number.

Resources: MIT 18.02 (multivariable calculus) or Stewart’s *Calculus*; 3Blue1Brown *Essence of Calculus*; MML Chapter 5; the *Matrix Cookbook* (reference only).

Deliverable: derive five gradients by hand and verify each with finite differences (central difference, error $\approx 10^{-6}$); implement gradient descent on a 2D quadratic and plot its path for several condition numbers.

4.4 Weeks 10–12 — Probability

Learn:

- Sample spaces, events, conditional probability, independence, Bayes’ rule, law of total probability.
- Discrete and continuous random variables, PMF/PDF/CDF, expectation, variance, covariance, correlation.
- Key distributions: Bernoulli, Binomial, Poisson, Geometric, Uniform, Exponential, Gaussian, Gamma, Beta, Dirichlet, Student- t .
- Joint, marginal and conditional distributions; the multivariate Gaussian (conditioning and marginalisation formulas).
- Law of large numbers; central limit theorem; Markov, Chebyshev and Hoeffding inequalities.
- Transformations of random variables; moment generating functions (briefly).

Resources: Blitzstein & Hwang, *Introduction to Probability*, together with Harvard Stat 110 lectures; MML Chapter 6; Wasserman, *All of Statistics*, Chapters 1–5.

Practice: at least 60 problems over the three weeks; simulate every result you prove (Monte Carlo).

Deliverable: (1) simulate the LLN and CLT for three different distributions and plot them; (2) derive the conditional distribution of a bivariate Gaussian and check it by simulation; (3) a 2-page L^AT_EX note on Bayes' rule with a medical-test example.

4.5 Weeks 13–14 — Statistics

Learn:

- Estimators, bias, variance, mean squared error, consistency, sufficiency (intuition).
- Maximum likelihood estimation, Fisher information, asymptotic normality; MAP estimation.
- Confidence intervals; hypothesis testing; p -values and what they do *not* mean; type I/II errors; power; multiple testing and the false discovery rate.
- The bootstrap and permutation tests.
- Frequentist versus Bayesian viewpoints, with one worked example each.
- Linear regression as a statistical model (Gaussian noise $\Rightarrow$ least squares = MLE).

Resources: Wasserman, *All of Statistics*, Chapters 6–13; MML Chapter 8; Casella & Berger (reference for rigorous proofs).

Derive by hand: MLE for Bernoulli, Gaussian, Poisson; show that least squares is the MLE under Gaussian noise; show the sample variance with divisor n is biased.

Deliverable: bootstrap confidence intervals for a median and for a correlation coefficient; a permutation test for the difference of two group means; Benjamini–Hochberg correction implemented by hand.

4.6 Week 15 — Optimisation and information theory; Phase 1 exam

Learn:

- Convex sets and functions; why local minima of convex functions are global.
- Lagrange multipliers, the KKT conditions (intuition and one worked example).
- Entropy, cross-entropy, KL divergence, mutual information; Jensen's inequality; the link between MLE and minimising KL divergence.

Resources: Boyd & Vandenberghe, *Convex Optimization*, Chapters 2–3 and 5.1–5.5; MML Chapter 7; Cover & Thomas, Chapter 2 (selected).

Phase 1 exam (self-administered): write 20 problems (5 each on linear algebra, calculus, probability, statistics), solve them closed-book in 3 hours, grade yourself, and fix every gap.

Deliverable: a 3-page L^AT_EX “math cheat-sheet” of the identities you will actually use, and your graded exam.

Exit criteria — do not move on until all are true

- ☐ You can derive the normal equations, the SVD–eigenvalue link, and the gradient of any quadratic or least-squares loss without notes.

- ☐ You can state and use Bayes' rule, the multivariate Gaussian conditioning formula, LLN and CLT.
- ☐ You can write the likelihood of a model, take the log, and find the MLE by calculus.
- ☐ You can explain why KL divergence is non-negative and why minimising cross-entropy equals maximum likelihood.
- ☐ You scored at least 80% on your closed-book Phase 1 exam.

5 Phase 2 — Core Machine Learning

Weeks 16–30

about 150 hours

Goal

Learn every standard supervised and unsupervised algorithm well enough to derive it, implement it from scratch in NumPy, tune it properly, and know when it fails. This is the heart of the roadmap.

Primary course and books for the whole phase (pick one lecture course and one book as your spine, and use the others for second opinions):

- **Lectures:** Stanford CS229 (Andrew Ng) or the Machine Learning Specialization on Coursera for a gentler start; Caltech *Learning From Data* (Abu-Mostafa) for the theory flavour; NPTEL *Introduction to Machine Learning* for an Indian-university perspective.
- **Books:** *An Introduction to Statistical Learning with Python* (James, Witten, Hastie, Tibshirani) for intuition; *The Elements of Statistical Learning* (Hastie, Tibshirani, Friedman) for depth; Bishop's *Pattern Recognition and Machine Learning* and Murphy's *Probabilistic Machine Learning: An Introduction* for the probabilistic view.

The “from-scratch” standard

For every algorithm in this phase: (1) derive the objective and update rule on paper; (2) implement it in NumPy as a class with `fit` and `predict`; (3) check against `scikit-learn` on the same data; (4) plot something that shows how it behaves (decision boundary, loss curve, learning curve). Store each in `code/from_scratch/`.

5.1 Week 16 — The ML problem and workflow

- ☐ Learn the vocabulary: supervised, unsupervised, semi-supervised, reinforcement learning; features, labels, hypothesis class, loss, empirical risk minimisation (ERM).
- ☐ Understand the train / validation / test split and *why* the test set is touched once.
- ☐ Understand overfitting and underfitting with a polynomial-regression example.
- ☐ Read Domingos, “A Few Useful Things to Know About Machine Learning” (2012) and Breiman, “Statistical Modeling: The Two Cultures” (2001).
- ☐ Learn the `scikit-learn` API: estimators; the `fit`, `predict` and `transform` methods; `Pipeline`; and `ColumnTransformer`.

Deliverable: an end-to-end baseline (load → split → preprocess → model → evaluate) on one

dataset, with a dummy-classifier baseline for comparison.

5.2 Week 17 — Linear regression

Learn and derive: the closed-form solution $\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}$; the geometric (projection) view; batch, stochastic and mini-batch gradient descent; feature scaling and why it speeds convergence; polynomial and interaction features; when $X^\top X$ is singular.

Implement: linear regression via (a) normal equations, (b) gradient descent, (c) SGD. Compare their solutions and speeds.

Analyse: learning curves (error vs. training-set size); residual plots; R^2 and its limits.

Deliverable: from-scratch linear regression, learning-curve plot, and a 1-page note on when each solver is preferable.

5.3 Week 18 — Bias–variance, regularisation, model selection

Learn and derive:

- The bias–variance decomposition: for squared loss,

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{(\mathbb{E}\hat{f}(x) - f(x))^2}_{\text{bias}^2} + \underbrace{\text{Var}(\hat{f}(x))}_{\text{variance}} + \underbrace{\sigma^2}_{\text{noise}}.$$

- Ridge regression: $\hat{\mathbf{w}}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$; its Bayesian interpretation (Gaussian prior); its effect through the SVD.
- Lasso, elastic net, and why the ℓ_1 penalty produces sparse solutions (geometry of the constraint set).
- Cross-validation: k -fold, stratified, grouped, leave-one-out, time-series splits. Nested CV for unbiased model selection.
- Hyperparameter search: grid, random, and Bayesian optimisation (e.g. Optuna).

Deliverable: regularisation-path plots for ridge and lasso; a nested-CV experiment; derivation of the ridge closed form in $\text{\LaTeX}$.

Common pitfall: data leakage

Fitting a scaler, imputer or feature selector on the *whole* dataset before splitting leaks information from the validation set into training. Always put preprocessing *inside* a Pipeline so it is re-fit within each CV fold.

5.4 Week 19 — Logistic regression and classification metrics

Learn and derive:

- The logistic (sigmoid) model $p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$; derive the loss as the negative log-likelihood and show its gradient is $X^\top (\hat{\mathbf{p}} - \mathbf{y})$.
- Convexity of the logistic loss; Newton’s method / IRLS; regularised logistic regression.
- Multiclass: softmax regression and one-vs-rest.

- Metrics: accuracy, precision, recall, F1, ROC-AUC, PR-AUC, log-loss, confusion matrix; threshold selection; why accuracy misleads on imbalanced data.

Deliverable: from-scratch logistic regression (gradient descent and Newton), with ℓ_2 regularisation; ROC and PR curves computed by hand and compared with `sklearn.metrics`.

5.5 Week 20 — Generative classifiers, k -NN, curse of dimensionality

Learn and derive: Naive Bayes (Gaussian and multinomial) and its independence assumption; Linear and Quadratic Discriminant Analysis; generative vs. discriminative models and when each wins; k -nearest neighbours, distance metrics, weighting; the curse of dimensionality (distances concentrate); KD-trees for fast search.

Deliverable: from-scratch Gaussian Naive Bayes, LDA and k -NN; a figure showing how k -NN accuracy degrades as you add irrelevant features.

5.6 Weeks 21–22 — Decision trees and bagging

Learn and derive:

- Impurity measures (entropy, Gini), information gain, the greedy CART algorithm for classification and regression.
- Handling categorical features, missing values, stopping rules, cost-complexity pruning.
- Why a single tree has high variance; bootstrap aggregation (bagging); random forests, out-of-bag error, feature importance (impurity-based and permutation).

Read: Breiman, “Random Forests” (2001).

Deliverable: from-scratch CART tree and random forest (bagging + random feature subsets); compare OOB error to cross-validated error.

5.7 Weeks 23–24 — Boosting and stacking

Learn and derive:

- AdaBoost (exponential loss view), gradient boosting as functional gradient descent; shrinkage, subsampling, tree depth, early stopping.
- Modern libraries: XGBoost, LightGBM, CatBoost — what each optimises differently, and the key hyperparameters.
- Stacking and blending; why diverse base models matter.

Read: Friedman, “Greedy Function Approximation: A Gradient Boosting Machine” (2001); Chen & Guestrin, “XGBoost” (KDD 2016).

Deliverable: from-scratch gradient boosting for squared loss with shallow regression trees; a tabular-data notebook that tunes XGBoost or LightGBM with proper nested validation.

5.8 Weeks 25–26 — Support vector machines and kernels

Learn and derive:

- The maximum-margin classifier; hard- and soft-margin primal; hinge loss.
- The Lagrangian dual: for the soft-margin SVM,

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(\mathbf{x}_i, \mathbf{x}_j) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0.$$

- KKT conditions and support vectors; the kernel trick; polynomial and RBF kernels; Mercer's condition; kernel ridge regression; support vector regression.
- Choosing C and γ ; why scaling matters for SVMs.

Read: Cortes & Vapnik, “Support-Vector Networks” (1995).

Deliverable: SVM trained via a QP solver on the dual (e.g. `cvxopt`) or a simplified SMO; decision-boundary plots for linear, polynomial and RBF kernels; a 2-page derivation note in $\text{\LaTeX}$.

5.9 Week 27 — Clustering and mixture models

Learn and derive: k -means (Lloyd's algorithm, k -means++ initialisation, convergence); hierarchical clustering and linkage; DBSCAN; Gaussian mixture models and the **EM algorithm** (derive the E-step and M-step); choosing k (elbow, silhouette, BIC); external vs. internal evaluation (ARI, NMI, silhouette).

Deliverable: from-scratch k -means and EM for a Gaussian mixture; a figure showing EM's log-likelihood increasing monotonically.

5.10 Week 28 — Dimensionality reduction and feature selection

Learn and derive: PCA from two viewpoints (maximum variance; minimum reconstruction error) and show they agree; PCA via SVD; explained-variance ratio; kernel PCA; t-SNE and UMAP and, crucially, *how not to over-interpret them*; NMF and ICA overview; feature selection by filter, wrapper and embedded (lasso, tree-based) methods.

Deliverable: from-scratch PCA; a notebook comparing PCA, t-SNE and UMAP on the same dataset and listing what each does and does not preserve.

5.11 Week 29 — Practical ML engineering

- ☐ Feature engineering: scaling, log transforms, binning, interactions, target and frequency encoding (done inside CV!), date/time features.
- ☐ Missing data: MCAR/MAR/MNAR; imputation strategies and their risks.
- ☐ Imbalanced data: class weights, resampling (SMOTE and its caveats), threshold tuning, appropriate metrics.
- ☐ Probability calibration: reliability diagrams, Platt scaling, isotonic regression.
- ☐ Interpretability: permutation importance, partial dependence, SHAP and LIME — and their failure modes (correlated features).
- ☐ Reproducibility: fixed seeds, saved splits, pinned environments.

Deliverable: a leakage-free Pipeline on a messy dataset, with a calibration plot and an interpretability report.

5.12 Week 30 — Phase 2 capstone and review

Capstone: choose one real dataset (see Section 8) and produce a full project: problem statement, EDA, baselines, at least five model families, nested-CV model selection, calibration, error analysis, interpretation, limitations. Write it up as a 6–8 page $\text{\LaTeX}$ report.

Review: re-do the three algorithms you found hardest, from a blank page.

Exit criteria — do not move on until all are true

- ☐ You have working from-scratch implementations of: linear and logistic regression, k -NN, Naive Bayes, a decision tree, random forest, gradient boosting, SVM (via the dual or SMO), k -means, a Gaussian mixture via EM, and PCA.
- ☐ You can derive the ridge solution, the logistic gradient, the SVM dual and the EM updates on paper.
- ☐ You can explain bias–variance, and diagnose over-/under-fitting from a learning curve.
- ☐ You can choose appropriate metrics and a leak-free validation scheme for a new problem.
- ☐ Your capstone report is complete, reproducible, and honest about limitations.

6 Phase 3 — Advanced Classical ML and Theory

Weeks 31–42

about 120 hours

Goal

Move from “using algorithms correctly” to “understanding why they work and when they provably do”. This phase covers statistical learning theory, convex optimisation, Bayesian and probabilistic ML, sparsity, online learning, and trustworthy ML. It is the bridge to reading research papers.

Mindset for this phase

Proofs are now the main activity. Read a theorem, close the book, and reconstruct the proof. When you get stuck, look at one line of the proof, then close the book again. Write the cleaned-up proofs in $\text{\LaTeX}$; this is excellent practice for paper writing.

6.1 Weeks 31–32 — Statistical learning theory

Learn:

- The PAC learning framework; realizable vs. agnostic settings; the No-Free-Lunch theorem.
- Finite hypothesis classes and the union bound; Hoeffding-based generalisation bounds.
- Growth function, VC dimension, Sauer’s lemma, the fundamental theorem of statistical learning.
- Rademacher complexity and data-dependent bounds; margin bounds for SVMs (intuition).
- Structural risk minimisation; the approximation–estimation trade-off.

Resources: Shalev-Shwartz & Ben-David, *Understanding Machine Learning: From Theory to Algorithms* (Chapters 2–6, 26, 28); Mohri, Rostamizadeh & Talwalkar, *Foundations of Machine Learning*

(Chapters 2–3).

Derive: for a finite class $\mathcal{H}$ and a loss in $[0, 1]$, with probability at least $1 - \delta$,

$$L_{\mathcal{D}}(h) \leq L_S(h) + \sqrt{\frac{\ln |\mathcal{H}| + \ln(1/\delta)}{2m}} \quad \text{for all } h \in \mathcal{H}.$$

Deliverable: a 4-page L^AT_EX note proving the finite-class bound and stating (with intuition) the VC and Rademacher bounds; a short simulation showing the bound is valid but loose.

6.2 Weeks 33–34 — Convex optimisation for ML

Learn:

- Convex analysis refresher; subgradients; Lagrangian duality, strong duality, KKT conditions in depth.
- Gradient descent convergence rates for convex, smooth, and strongly convex functions.
- Stochastic gradient descent: variance, step-size schedules, convergence in expectation.
- Momentum, Nesterov acceleration, Newton and quasi-Newton (L-BFGS), coordinate descent.
- Proximal methods and ISTA/FISTA for ℓ_1 -regularised problems; ADMM (overview).

Resources: Boyd & Vandenberghe, *Convex Optimization* (Chapters 3–5, 9); Bubeck, *Convex Optimization: Algorithms and Complexity*; Nocedal & Wright (reference).

Deliverable: implement GD, SGD, momentum, Nesterov and L-BFGS on the same logistic-regression problem and plot convergence in iterations *and* in wall-clock time; implement ISTA for the lasso and compare with `sklearn`.

6.3 Weeks 35–36 — Bayesian ML and Gaussian processes

Learn and derive:

- Bayesian inference: prior, likelihood, posterior, posterior predictive; conjugate pairs (Beta–Bernoulli, Gamma–Poisson, Normal–Normal).
- Bayesian linear regression: derive the posterior $\mathcal{N}(\mathbf{m}_N, S_N)$ and predictive distribution; marginal likelihood (evidence) and Occam’s razor.
- Gaussian processes: the function-space view, kernels (RBF, Matérn, periodic), hyperparameter learning by marginal likelihood, GP classification (Laplace approximation, overview).
- Bayesian model comparison, BIC, and the relationship between ridge regression, kernel ridge regression and GP regression.

Resources: Bishop PRML Chapters 1–3 and 6; Rasmussen & Williams, *Gaussian Processes for Machine Learning* (Chapters 1–2, 4–5); Murphy, *Probabilistic Machine Learning: An Introduction* (the relevant chapters).

Deliverable: from-scratch Bayesian linear regression and GP regression with uncertainty bands; a comparison of marginal-likelihood hyperparameter fitting against cross-validation.

6.4 Weeks 37–38 — Graphical models and approximate inference

Learn and derive:

- Directed (Bayesian networks) and undirected (Markov random fields) models; conditional independence, d -separation.
- Exact inference: variable elimination, belief propagation on trees.
- Hidden Markov models: forward–backward, Viterbi, Baum–Welch (EM).
- EM in general: the ELBO decomposition and why EM increases the likelihood.
- Variational inference: mean-field approximation, coordinate ascent.
- MCMC: Metropolis–Hastings, Gibbs sampling, diagnostics (trace plots, $\hat{R}$, effective sample size); Hamiltonian Monte Carlo (concept).

Read: Dempster, Laird & Rubin, “Maximum Likelihood from Incomplete Data via the EM Algorithm” (1977); Blei, Ng & Jordan, “Latent Dirichlet Allocation” (2003).

Resources: Koller & Friedman, *Probabilistic Graphical Models* (selected chapters); Bishop PRML Chapters 8–11.

Deliverable: an HMM implemented from scratch (forward–backward + Baum–Welch) on a toy sequence task; a Gibbs sampler for a simple model (e.g. a mixture or a change-point model) with convergence diagnostics.

6.5 Week 39 — Sparsity and high-dimensional statistics

Learn: the $p \gg n$ regime; the lasso’s behaviour and conditions for support recovery (irrepresentable condition, restricted eigenvalue — at an intuition level); stability selection; compressed-sensing intuition; multiple hypothesis testing, FWER vs. FDR, Benjamini–Hochberg and its assumptions.

Resources: Hastie, Tibshirani & Wainwright, *Statistical Learning with Sparsity*; ESL Chapters 3 and 18; Tibshirani, “Regression Shrinkage and Selection via the Lasso” (1996).

Why it matters: high-dimensional measurements with few samples (e.g. gene expression with tens of thousands of features and tens or hundreds of subjects) are the normal case in biomedical data, and naive feature selection there is a major source of irreproducible results.

Deliverable: a simulation showing how lasso support recovery depends on n , p , sparsity and correlation; selection bias demonstration (selecting features on the full data before CV inflates accuracy on pure noise).

6.6 Week 40 — Online learning and bandits

Learn: regret; the perceptron mistake bound; Hedge / multiplicative weights; online gradient descent and its regret bound; multi-armed bandits — ϵ -greedy, UCB, Thompson sampling; contextual bandits (overview).

Read: Auer, Cesa-Bianchi & Fischer, “Finite-time Analysis of the Multiarmed Bandit Problem” (2002).

Deliverable: a bandit simulation comparing ϵ -greedy, UCB and Thompson sampling with regret curves averaged over many runs.

6.7 Week 41 — Trustworthy ML: uncertainty, shift, fairness, causality

- **Uncertainty:** calibration, prediction intervals, and **conformal prediction**. Read Angelopoulos & Bates, “A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty

Quantification” (2021).

- ☐ **Distribution shift:** covariate shift, label shift, concept drift; importance weighting; how to detect shift with a classifier two-sample test.
- ☐ **Fairness:** group metrics (demographic parity, equalised odds), their incompatibilities, and the limits of purely statistical fairness. Read Hardt, Price & Srebro, “Equality of Opportunity in Supervised Learning” (2016).
- ☐ **Causality primer:** correlation vs. causation, confounding, potential outcomes, DAGs, adjustment, propensity scores. Resource: Hernán & Robins, *Causal Inference: What If* (free).

Deliverable: split-conformal prediction on a regression and a classification task, with empirical coverage checked on held-out data; a short note on a case where a model’s accuracy hides a failure under distribution shift.

6.8 Week 42 — Phase 3 integration and review

- ☐ Choose two topics from this phase and write an 8–10 page survey-style note on each (what is known, what is open, key papers).
- ☐ Re-prove from memory: the finite-class generalisation bound, the convergence of GD on a smooth convex function, and the Bayesian linear regression posterior.
- ☐ Identify the three topics that interest you most — these feed into your research question in Phase 4.

Exit criteria — do not move on until all are true

- ☐ You can prove a finite-class generalisation bound and explain what VC dimension and Rademacher complexity measure.
- ☐ You can state the convergence rate of GD/SGD under convexity and strong convexity, and implement proximal gradient for the lasso.
- ☐ You can derive Bayesian linear regression and explain how GPs extend it; you can run and diagnose an MCMC sampler.
- ☐ You understand EM as maximising the ELBO and can implement it for a mixture model and an HMM.
- ☐ You can construct a conformal prediction interval and verify its coverage empirically.
- ☐ You have written at least two clean L^AT_EX proofs or survey notes.

7 Phase 4 — Research Skills

Weeks 43–52

about 100 hours

Goal

Turn knowledge into research ability: read papers critically, reproduce results, design rigorous experiments, pick a question, carry out a small original project, write it up, and make your work visible to people who offer research internships.

7.1 Week 43 — Reading papers

Method: use the three-pass approach from S. Keshav, “How to Read a Paper”.

1. **Pass 1 (5–10 min):** title, abstract, introduction, section headings, conclusions. Decide if it is worth more time.
2. **Pass 2 (about 1 hour):** read the whole paper, skip proofs; study figures and tables; list the main claims and the evidence given.
3. **Pass 3 (several hours):** re-derive key steps; challenge every assumption; note what you would do differently.

Set up your tools: Zotero for references; a notes template (claim, method, evidence, assumptions, weaknesses, ideas); search with Google Scholar, Semantic Scholar and arXiv; follow citations forwards and backwards from a good paper.

Where ML work appears: journals — JMLR, TMLR, Machine Learning, Bioinformatics; conferences — ICML, NeurIPS, COLT, AISTATS, UAI, KDD, ICLR; and application venues in your domain.

Read and summarise three papers (one page each, with a “what I doubt” section): Breiman (2001) *Two Cultures*; Breiman (2001) *Random Forests*; Chen & Guestrin (2016) *XGBoost*.

Deliverable: three one-page summaries in your notes folder, plus a Zotero library with at least 20 papers on your topics of interest.

7.2 Weeks 44–45 — Reproduce a published result

- ☐ Pick a classical, well-cited paper with a public dataset and clearly reported numbers (examples: lasso on a standard dataset; random-forest vs. SVM comparisons; a conformal prediction experiment; a boosting variant).
- ☐ Write down the target: which table or figure will you reproduce, and what counts as “matching”?
- ☐ Implement the method from the paper, *not* from the authors’ code, then compare to released code if it exists.
- ☐ Record every discrepancy and hypothesise its cause (preprocessing, split, random seed, hyper-parameters, a paper typo).
- ☐ Run each experiment with at least 10 random seeds; report mean $\pm$ standard deviation.

Deliverable: a public repository and a 4-page L^AT_EX reproduction report: what matched, what did not, and why.

7.3 Week 46 — Experimental rigour

Learn and apply:

- **Baselines:** always include a trivial baseline and a strong, well-tuned simple baseline (often a regularised linear model or gradient-boosted trees).
- **Ablations:** remove or change one component at a time to show which part matters.
- **Statistics:** multiple seeds; confidence intervals via bootstrap; paired tests; for comparing several algorithms across datasets, read Demšar, “Statistical Comparisons of Classifiers over Multiple Data Sets” (JMLR 2006) and Dietterich (1998) on approximate tests.

- **Fair tuning:** give every method the same tuning budget; report the search space.
- **Data handling:** freeze splits, avoid leakage, document preprocessing, keep the test set untouched until the final run.
- **Tooling:** experiment tracking (MLflow or Weights & Biases), configuration management (Hydra or YAML files), data versioning (DVC), pinned environments.

Deliverable: a reusable experiment harness: config file → seeded runs → logged metrics → aggregated table with confidence intervals.

7.4 Week 47 — Finding a research question

- ☐ List your three favourite topics from Phase 3. For each, read one recent survey and the “future work” sections of five papers.
- ☐ Look for gaps of these kinds: (a) a method that has never been compared fairly to a strong baseline on some data type; (b) a known assumption that is rarely checked; (c) a combination of two existing ideas; (d) a failure case that is reported but not explained; (e) an application in your own domain where standard methods are used without validation.
- ☐ Write three candidate questions in the form: “Does X improve Y under condition Z , compared to W ?”
- ☐ For each question, write one paragraph on how you would answer it, what data you need, and what result would count as a negative outcome (negative results are fine).
- ☐ Discuss your list with a professor, senior student, or the club mentor, and choose *one* question that fits 3 weeks of work.

Deliverable: a one-page proposal: question, motivation, related work (5–10 papers), method, datasets, baselines, evaluation plan, risks.

7.5 Weeks 48–50 — Mini research project

Week 48: literature review (write 1–2 pages of related work), implement baselines, build the experiment harness for your question.

Week 49: implement the proposed variant or analysis; run the main experiments; check sanity cases (does it work on synthetic data where you know the answer?).

Week 50: ablations, error analysis, failure cases, statistical tests; decide what your honest conclusion is. If results are negative or mixed, *report that*.

Common pitfall: unreported researcher degrees of freedom

Trying many variants and reporting only the best one inflates results. Keep a log of *every* experiment, decide your primary metric before running, and correct for multiple comparisons when you test many hypotheses.

Deliverable: a clean repository, all raw results, and figures generated by scripts (not by hand).

7.6 Week 51 — Writing the paper

Structure: title; abstract (problem, method, result, claim in 150–200 words); introduction (motivation, gap, contributions as a bullet list); related work; method; experiments (setup, baselines, results, ablations); discussion and limitations; conclusion; references.

- Use an official conference or workshop L^AT_EX template (for example the NeurIPS, ICML or ICLR style files) so you get used to page limits and formatting.
- Use BibTeX; every claim needs either a citation, a proof, or an experiment.
- Figures: vector graphics (PDF), readable fonts, colour-blind-safe palettes, informative captions that stand alone.
- Tables: bold the best result *only* if differences are statistically meaningful; show standard deviations.
- Do a “peer review” of your own paper: write the three strongest rejection reasons and address them.

Deliverable: a 6–8 page paper draft in L^AT_EX, reviewed by at least one other person.

7.7 Week 52 — Release, outreach, next steps

- ☐ Clean and release the repository: README with exact commands to reproduce each figure; license; pinned environment; archived release (Zenodo gives you a DOI).
- ☐ Post the paper on arXiv if you have a supervisor’s agreement and meet the endorsement requirements, or submit to a student or workshop track. Write a short blog post summarising the work for non-experts.
- ☐ Update your CV (education, projects, links) and create a one-page “research portfolio” PDF.
- ☐ **Internship search:** make a list of 20 research groups whose work matches your interests (IITs, IISc, other Indian institutes, and international labs). Read two of each group’s recent papers before writing to them. Check current opportunities and deadlines for programs such as the Indian Academy of Sciences Summer Research Fellowship, IISc summer programs, DAAD WISE, Mitacs Globalink, and Google Summer of Code; eligibility and dates change every year, so verify them on the official pages.
- ☐ Use the cold-email template in Appendix D. Send in batches of five, then follow up once after 10 days.
- ☐ Start the Deep Learning & Neural Networks roadmap (see Section 10).

Exit criteria — research level reached when all are true

- ☐ You have reproduced at least one published result and can explain every discrepancy.
- ☐ You have completed a small original study with baselines, ablations and statistics, and written it as a paper.
- ☐ You can read a new ML paper on a familiar topic in a few hours and write a fair critique.
- ☐ You have a public repository and written report that someone else can run and check.
- ☐ You have contacted at least 10 research groups or applied to at least 3 programs.

8 Projects Across the Roadmap

Projects are where learning becomes evidence. Every project should have: a clear question, a reproducible repository, baselines, honest evaluation, and a short $\text{\LaTeX}$ report.

8.1 Project ladder

Tier	When	What you produce
Warm-up	Weeks 1–15	EDA notebook; from-scratch numerical routines (Gaussian elimination, power iteration, gradient descent); simulation studies of LLN, CLT, bootstrap.
Core capstone	Week 30	An end-to-end tabular ML project with five model families, nested CV, calibration, interpretation, and a 6–8 page report.
Advanced study	Weeks 31–42	Short theory-flavoured notes and simulations (generalisation bounds, lasso support recovery, conformal coverage, bandit regret).
Mini research	Weeks 47–51	A small original study with a proposal, baselines, ablations, statistics, and a paper draft.

8.2 Core capstone: choose one dataset (Week 30)

Pick a dataset with at least a few thousand rows (or a justified smaller one) and a clear prediction target.

General tracks

- Housing prices (regression, mixed feature types, missing values).
- Credit default or customer churn (classification, class imbalance, calibration matters).
- A Kaggle “tabular playground”-style competition (benchmarked against a public leaderboard).

Biology and biomedical track (optional)

- Cancer subtype or tissue classification from public gene-expression data (NCBI GEO: <https://www.ncbi.nlm.nih.gov/geo/>); the main challenge is $p \gg n$, so nested CV and feature-selection leakage are central.
- Protein property or function prediction from sequence-derived features (amino-acid composition, physicochemical descriptors) using public databases such as UniProt.
- Clinical tabular data from open repositories (UCI, OpenML) to study calibration and subgroup performance.
- Classification of physiological or wearable-sensor signals using hand-crafted time- and frequency-domain features and classical models.

Data access

Use open datasets. Avoid controlled-access biomedical data unless you have formal permission, and never upload personal health data to public repositories. Always read the dataset licence and cite the source.

8.3 Mini-research ideas (Weeks 47–51)

These are *templates* for questions; refine one with your mentor so it is both feasible and useful.

1. **Fair benchmarking of feature selection** on several high-dimensional datasets, with strictly nested validation and stability metrics. Does the popular method survive proper evaluation?
2. **Simple vs. complex models on small data.** When do regularised linear models match or beat gradient-boosted trees as sample size shrinks? Plot performance against n for many datasets with confidence intervals.
3. **Calibration under shift.** How do the calibration and accuracy of classical models degrade when train and test come from different sources, and does recalibration help?
4. **Conformal prediction in practice.** Check empirical coverage of split conformal methods under mild distribution shift and under strong class imbalance; propose and test a simple fix.
5. **Reproducibility audit.** Re-run a published classical-ML comparison with proper tuning budgets and multiple seeds; report which conclusions are robust.

Common pitfall: novelty obsession

A careful, honest negative or “it depends” result with strong baselines is more valuable (and more publishable at workshops) than a vague claim of a new method. Start by making the evaluation trustworthy.

9 Master Resource List

Choose *one* primary resource per topic and finish it; do not collect more than you can use. Items marked ★ are freely available from the authors or publishers.

9.1 Mathematics

Topic	Primary resources
Linear algebra	Strang, <i>Introduction to Linear Algebra</i> and MIT 18.06 lectures; 3Blue1Brown <i>Essence of Linear Algebra</i> ; Axler, <i>Linear Algebra Done Right</i> (proof-based alternative).
Calculus	MIT 18.01 / 18.02; Stewart, <i>Calculus</i> ; 3Blue1Brown <i>Essence of Calculus</i> .
Combined maths	ML Deisenroth, Faisal & Ong, <i>Mathematics for Machine Learning</i> ★ (https://mml-book.com).
Probability	Blitzstein & Hwang, <i>Introduction to Probability</i> ★; Harvard Stat 110 lectures.
Statistics	Wasserman, <i>All of Statistics</i> ; Casella & Berger, <i>Statistical Inference</i> (rigorous reference).
Optimisation	Boyd & Vandenberghe, <i>Convex Optimization</i> ★; Bubeck, <i>Convex Optimization: Algorithms and Complexity</i> ★.
Information theory	Cover & Thomas, <i>Elements of Information Theory</i> ; MacKay, <i>Information Theory, Inference, and Learning Algorithms</i> ★.

9.2 Machine learning books

Book	Best used for
James, Witten, Hastie, Tibshirani, <i>An Introduction to Statistical Learning (with Python)</i> *	Intuition-first first pass; excellent labs.
Hastie, Tibshirani, Friedman, <i>The Elements of Statistical Learning</i> *	Depth and rigour on regression, trees, boosting, SVMs, high dimensions.
Bishop, <i>Pattern Recognition and Machine Learning</i>	Probabilistic viewpoint; graphical models; Bayesian methods.
Murphy, <i>Probabilistic Machine Learning: An Introduction (and Advanced Topics)</i> *	Modern, broad probabilistic treatment.
Shalev-Shwartz & Ben-David, <i>Understanding Machine Learning: From Theory to Algorithms</i> *	Learning theory (PAC, VC, Rademacher).
Mohri, Rostamizadeh, Talwalkar, <i>Foundations of Machine Learning</i> *	Theory, kernels, bounds.
Rasmussen & Williams, <i>Gaussian Processes for Machine Learning</i> * (https://gaussianprocess.org/gpml/)	Gaussian processes.
Koller & Friedman, <i>Probabilistic Graphical Models</i>	Graphical models reference.
Hastie, Tibshirani, Wainwright, <i>Statistical Learning with Sparsity</i> *	Lasso and sparsity.
Hernán & Robins, <i>Causal Inference: What If</i> *	Causal inference primer.
Géron, <i>Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow</i>	Practical workflows (use only the classical chapters for now).
VanderPlas, <i>Python Data Science Handbook</i> *	NumPy, pandas, Matplotlib, scikit-learn in practice.

9.3 Lecture courses

- Stanford CS229 *Machine Learning* (Andrew Ng; lecture notes are excellent for derivations).
- Caltech *Learning From Data* (Yaser Abu-Mostafa) — compact and theory-aware.
- Carnegie Mellon 10-701 *Introduction to Machine Learning* — broader, more statistical.
- NPTEL *Introduction to Machine Learning* (IIT Madras and IIT Kharagpur offerings) — a good

fit for Indian-university schedules.

- Coursera *Machine Learning Specialization* (Andrew Ng) — a gentle first pass.
- MIT 18.06 (linear algebra), Harvard Stat 110 (probability).

9.4 Landmark papers (read in roughly this order)

Phase	Author(s), year	Title
2	Domingos, 2012	A Few Useful Things to Know About Machine Learning
2	Breiman, 2001	Statistical Modeling: The Two Cultures
2	Tibshirani, 1996	Regression Shrinkage and Selection via the Lasso
2	Zou & Hastie, 2005	Regularization and Variable Selection via the Elastic Net
2	Breiman, 2001	Random Forests
2	Freund & Schapire, 1997	A Decision-Theoretic Generalization of On-line Learning and an Application to Boosting
2	Friedman, 2001	Greedy Function Approximation: A Gradient Boosting Machine
2	Cortes & Vapnik, 1995	Support-Vector Networks
2	Chen & Guestrin, 2016	XGBoost: A Scalable Tree Boosting System
2	Ke et al., 2017	LightGBM: A Highly Efficient Gradient Boosting Decision Tree
2	Pedregosa et al., 2011	Scikit-learn: Machine Learning in Python
3	Valiant, 1984	A Theory of the Learnable
3	Dempster, Laird & Rubin, 1977	Maximum Likelihood from Incomplete Data via the EM Algorithm
3	Blei, Ng & Jordan, 2003	Latent Dirichlet Allocation
3	Rahimi & Recht, 2007	Random Features for Large-Scale Kernel Machines
3	Auer, Cesa-Bianchi & Fischer, 2002	Finite-time Analysis of the Multiarmed Bandit Problem
3	Ribeiro, Singh & Guestrin, 2016	“Why Should I Trust You?”: Explaining the Predictions of Any Classifier
3	Lundberg & Lee, 2017	A Unified Approach to Interpreting Model Predictions
3	Hardt, Price & Srebro, 2016	Equality of Opportunity in Supervised Learning
3	Angelopoulos & Bates, 2021	A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification
4	Dietterich, 1998	Approximate Statistical Tests for Comparing Supervised Classification Learning Algorithms
4	Demšar, 2006	Statistical Comparisons of Classifiers over Multiple Data Sets
4	Keshav, 2007	How to Read a Paper

9.5 Tools and libraries

Purpose	Tools
Core scientific Python	NumPy, SciPy, pandas, Matplotlib, seaborn
Classical ML	scikit-learn (https://scikit-learn.org), XGBoost, LightGBM, CatBoost
Statistics / Bayesian	statsmodels, PyMC, emcee, Stan (optional)
Optimisation	SciPy optimize, cvxpy / cvxopt, Optuna for hyperparameters
Interpretability	SHAP, LIME, scikit-learn inspection tools
Experiment management	MLflow, Weights & Biases, Hydra, DVC, pytest
Writing & references	Overleaf or local $\text{\LaTeX}$, BibTeX, Zotero
Paper discovery	arXiv, Google Scholar, Semantic Scholar

9.6 Datasets and practice platforms

- **UCI Machine Learning Repository** (<https://archive.ics.uci.edu>) and **OpenML** (<https://www.openml.org>) — classic, well-documented benchmarks.
- **Kaggle** (<https://www.kaggle.com>) — datasets, notebooks, and competitions for practice against a leaderboard.
- **NCBI GEO** (<https://www.ncbi.nlm.nih.gov/geo/>) — public gene-expression datasets for the biology track.
- **scikit-learn built-in datasets** — quick debugging data for from-scratch implementations.

10 Ready for Deep Learning and Neural Networks?

Deep learning builds directly on classical ML. If the foundations below are solid, the DL roadmap will feel like a natural extension rather than a new subject.

10.1 Why classical ML first pays off

Classical idea you already own	Where it reappears in deep learning
Logistic / softmax regression	A single neuron / the output layer of a classifier.
Chain rule and matrix calculus	Backpropagation <i>is</i> the chain rule applied systematically.
Gradient descent, SGD, momentum, ℓ_2 penalty	Adam, learning-rate schedules, weight decay.
Bias–variance, cross-validation, leakage	Overfitting, early stopping, honest validation of large models.
PCA and the SVD	Linear autoencoders, embeddings, low-rank adaptation ideas.
Mixture models, EM, variational inference	Variational autoencoders and other generative models.
Kernels and Gaussian processes	Neural tangent kernels, uncertainty in deep models.
Calibration, conformal prediction	Trustworthy deep models and uncertainty estimation.
Experiment rigour and baselines	Fair comparison with strong, well-tuned classical baselines.

10.2 Readiness checklist

Move on when you can answer “yes” to every item. If not, revisit the indicated phase.

- ☐ (Phase 1) I can compute the gradient of a composite function using the chain rule and matrix calculus, and verify it numerically.
- ☐ (Phase 2) I have implemented logistic regression with gradient descent from scratch and understand each line of its update.
- ☐ (Phase 2) I can diagnose over-fitting and under-fitting from learning curves and choose regularisation accordingly.
- ☐ (Phase 2) I can set up a leak-free validation scheme for a new dataset without help.
- ☐ (Phase 3) I can analyse the convergence of gradient descent on a convex function and explain the role of step size.
- ☐ (Phase 3) I can explain the ELBO and why EM / variational inference works.
- ☐ (Phase 4) I have a working experiment harness (config, seeds, logging, aggregation) that I can reuse.
- ☐ (Phase 4) I have written at least one full paper-style report in $\text{\LaTeX}$.
- ☐ (All) I have a strong classical baseline habit: before any complex model, I run a tuned linear model and gradient-boosted trees.

10.3 Preview: what the next roadmap will cover

The Deep Learning & Neural Networks roadmap should be planned as a separate document with the same structure (skip tests, weekly deliverables, exit criteria). A natural sequence is:

1. Neural network fundamentals: perceptrons, MLPs, backpropagation from scratch, initialisation, regularisation, optimisers.
2. A deep-learning framework (PyTorch): tensors, autograd, training loops, debugging.
3. Convolutional networks and computer vision basics.
4. Sequence models, attention, and Transformers.
5. Generative models and representation learning.
6. Large-scale training practice, evaluation, and reproducibility.
7. Research topics in deep learning, including reading and reproducing recent papers.

When you are ready, ask for that roadmap and say which of your Phase 3 interests you want it to connect to.

A Master Week-by-Week Schedule

Tick each row only when the deliverable exists in your repository.

Week	Focus	Deliverable
Phase 0 — Setup and programming		
1	Shell, Git, Python environment, repository	Repo online; environment reproducible
2	Python core, NumPy, vectorisation	<code>utils.py</code> with five tested functions
3	pandas, plotting, EDA, first L ^A T _E X note	EDA notebook and one-page PDF
Phase 1 — Mathematics		
4–5	Linear algebra I: vectors, matrices, systems, rank	Gaussian elimination from scratch
6–7	Linear algebra II: projections, eigen, SVD	Power iteration; PCA via SVD
8–9	Multivariate and matrix calculus; gradient descent	Gradients checked numerically
10–12	Probability: distributions, Gaussian, LLN/CLT	Simulations; Bayes' rule note
13–14	Statistics: MLE, intervals, tests, bootstrap	Bootstrap CIs; BH procedure
15	Optimisation, information theory; Phase 1 exam	Graded exam; math cheat-sheet
Phase 2 — Core machine learning		
16	ML workflow, scikit-learn API	End-to-end baseline
17	Linear regression: closed form, GD, SGD	From-scratch regression; learning curves

continued on next page

Week	Focus	Deliverable
18	Bias–variance, ridge/lasso, cross-validation	Regularisation paths; nested CV
19	Logistic regression, metrics	From-scratch logistic; ROC/PR by hand
20	Naive Bayes, LDA, k -NN, curse of dimensionality	Three classifiers from scratch
21–22	Decision trees, bagging, random forests	CART and random forest from scratch
23–24	AdaBoost, gradient boosting, XGBoost/LightGBM, stacking	Gradient boosting from scratch; tuned GBM notebook
25–26	SVMs, duality, kernels	SVM via dual QP/SMO; derivation note
27	k -means, hierarchical, DBSCAN, GMM & EM	k -means and EM from scratch
28	PCA, t-SNE/UMAP, feature selection	PCA from scratch; comparison notebook
29	Feature engineering, imbalance, calibration, interpretability	Leak-free pipeline and calibration plot
30	Capstone project and review	6–8 page $\text{\LaTeX}$ report
Phase 3 — Advanced classical ML and theory		
31–32	Statistical learning theory: PAC, VC, Rademacher	Proof note; bound simulation
33–34	Convex optimisation: SGD, acceleration, proximal methods	Optimiser comparison; ISTA for lasso
35–36	Bayesian linear regression, Gaussian processes	BLR and GP from scratch
37–38	Graphical models, HMMs, EM, VI, MCMC	HMM from scratch; Gibbs sampler
39	Sparsity, high-dimensional statistics, FDR	Lasso recovery simulation; selection-bias demo
40	Online learning and bandits	Bandit regret curves
41	Conformal prediction, shift, fairness, causality primer	Conformal coverage experiments
42	Integration and review	Two survey-style notes; re-proofs
Phase 4 — Research skills		
43	Reading papers; Zotero; three summaries	Three one-page paper summaries
44–45	Reproduce a published result	Reproduction repo and report
46	Experimental rigour and tooling	Reusable experiment harness

continued on next page

Week	Focus	Deliverable
47	Choose a research question	One-page proposal
48–50	Mini research project	Repo, raw results, scripted figures
51	Write the paper in $\text{\LaTeX}$	6–8 page reviewed draft
52	Release, outreach, DL transition	Public release; emails/applications sent

B Suggested Repository Structure

```

ml-journey/
|-- README.md           # goals, progress table, how to run
|-- environment.yml      # or requirements.txt
|-- notes/
|   |-- latex/          # weekly notes, proofs, cheat-sheets
|   '-- error_log.md
|-- code/
|   |-- from_scratch/    # linear_regression.py, svm.py, em_gmm.py ...
|   |-- utils/
|   '-- tests/           # compare against scikit-learn
|-- projects/
|   |-- 00_eda/
|   |-- 01_capstone_core_ml/
|   '-- 02_mini_research/
|-- papers/
|   |-- summaries/
|   '-- reproductions/
'-- logs/
    '-- weekly/          # one file per week

```

C Weekly Log Template

Copy this into `logs/weekly/week_NN.md` every Sunday (10–15 lines is enough).

- Planned vs. done:** what the schedule said, what you actually completed.
- Hours spent:** theory / coding / project / review.
- One thing I can now do** that I could not last week.
- Hardest concept** and the exact point where I got stuck.
- Errors for the log:** mistakes in derivations or bugs, with causes.
- Plan for next week:** deliverable, and the first task for Monday.

D Cold-Email Template for Research Groups

Keep it under 150 words. Attach a one-page CV and link your repository.

Subject: Research internship enquiry — [your topic of interest]

Dear Prof. [Surname],

I am a [year] undergraduate at [institute] studying [programme]. I have completed a self-directed 12-month program in classical machine learning, including from-scratch implementations of core algorithms and a reproduction of [paper]. I read your recent papers [Title 1] and [Title 2], and I am especially interested in [one specific question or result].

I am looking for a research internship in [period]. A short example of my work is here: [link to repository or report]. I would be grateful for any opportunity to contribute, even in a supporting role.

Thank you for your time and consideration.

Sincerely,

[Name], [institute], [link]

Making the email work

Mention one *specific* detail from the professor's work (this shows you read it). Send to professors whose recent work you actually understand, follow up once after about 10 days, and thank people even when they decline.

E From-Scratch Implementation Checklist

Tick each when the implementation exists, is tested against `scikit-learn`, and has a plot.

Phase 1–2

- ☐ Gaussian elimination
- ☐ Power iteration; PCA via SVD
- ☐ Linear regression (normal eq., GD, SGD)
- ☐ Ridge and lasso (coordinate descent)
- ☐ Logistic regression (GD and Newton)
- ☐ Gaussian Naive Bayes, LDA
- ☐ k -NN with KD-tree option
- ☐ Decision tree (CART)
- ☐ Random forest
- ☐ Gradient boosting (squared loss)
- ☐ SVM (dual QP or SMO), kernels
- ☐ k -means and k -means++
- ☐ Gaussian mixture via EM
- ☐ Cross-validation and metrics (ROC, PR)

Phase 3

- ☐ SGD, momentum, Nesterov, L-BFGS comparison
- ☐ ISTA for lasso
- ☐ Bayesian linear regression
- ☐ Gaussian process regression
- ☐ HMM (forward–backward, Baum–Welch)
- ☐ Metropolis–Hastings and Gibbs samplers
- ☐ Variational inference for a small model
- ☐ Thompson sampling and UCB bandits
- ☐ Split conformal prediction
- ☐ Benjamini–Hochberg procedure

Consistency beats intensity. Ten honest hours a week for a year will take you further than any sprint.